Lab 09 · OOP 2

실습 · Notion 아카이브

Operator Overloading(연산자 오버로딩)

- `acc1 += 1000`
- > acc1 = acc1.__iadd__(1000)
- 이런 식으로 작동함
  - return self
  - 를 이용해서 self를 반환해 줘야 함

실습 1

'은행 계좌에서 Operator Overloading'

```
class BankAccount():
    def __init__(self, name=None, balance=0):
        self.name = name
        self.balance = balance

    def __add__(self, amount):
        self.balance += amount
        return self

    def __iadd__(self, amount):
        self.balance += amount
        return self

    def __sub__(self, amount):
        if self.balance >= amount:
            self.balance -= amount
            return self
        else:
            print('잔액 부족')
            return self

    def __isub__(self, amount):
        if self.balance >= amount:
```

```
            self.balance -= amount
            return self
        else:
            print('잔액 부족')
            return self

    def __str__(self):
        return f'name: {self.name}, balance: {self.balance}'

if __name__ == "__main__":
    acc1 = BankAccount("홍길동", 1000)
    print(acc1)
    acc1 + 1000
    print(acc1)
    acc1 += 1000
    print(acc1)
    acc1 - 1000
    print(acc1)
    acc1 -= 4000
    print(acc1)
    acc1 -= 1000
    print(acc1)
```

Inheritance(상속)

```
class DerivedClassName(BaseClassName):
<statement-1>
...
<statement-N>
```

- ⇒ 기존 **Class(클래스)**의 속성을 물려 받아 새로운 클래스를 만드는 것
  - 부모 클래스의 메서드와 클래스 속성은 자식 클래스에서 상속받아 사용할 수 있다. 부모 클래스의 `__init__()`에서 설정하는 인스턴스 속성까지 초기화하려면, 자식 클래스에서 `__init__()`을 재정의한 경우 `super().__init__(...)`을 호출해야 한다.
  - 다중 클래스를 상속받을 수 있음

```
class A:
    a = 1
    def print_a(self):
        print("A Class")
```

```
class B:
    b = 1
    def print_b(self):
        print("B Class")

class C(A):
    c = 1
    def print_c(self):
        print("C Class")

class D(C,B):
    d = 1
    def print_d(self):
        print("D Class")

c = C()
d = D()

'mro : 메서드 해석 순서 반환 함수'

print(D.mro())
d.print_a()
d.print_b()
d.print_c()
d.print_d()
```

- [<class '**main**.D'>, <class '**main**.C'>, <class '**main**.A'>, <class '**main**.B'>, <class '**Object(객체)**'>]
  A Class
  B Class
  C Class
  D Class

Method Overriding

- 기존 클래스 메서드를 상속 클래스에서 재정의 하는 것
- 기존 클래스의 메서드를 사용하고자 하는 경우 super() 사용

```
class A:
    def __init__(self):
        print("this is [A] class and [__init__] function")
```

```
class B(A):
    def __init__(self):
        super().__init__()
        print("this is [b] class and [__init__] function")

a = A()
b = B()
```

- this is [A] class and [**init**] **Function(함수)**
  this is [A] class and [**init**] function
  this is [b] class and [**init**] function

실습 1 상속 및 오버라이딩 [사람 ⇒ 학생]

- 사람 → 학생 + sid(학번) 기능

```
class Person:

    def __init__(self, name):
        self.name = name

    def introduce(self):
        print(f'안녕하세요, 저는 {self.name}입니다.')

class Student(Person):

    def __init__(self, name, sid):
        super().__init__(name)   # 필요한 변수 입력해줘야 함
        self.sid = sid

    def introduce(self):
        super().introduce()
        print(f'학번: {self.sid}')



# 테스트 코드
p = Person("고길동")
p.introduce()
print()
```

```
s = Student("박희동", 20250001)
s.introduce()
# 실행 결과
안녕하세요, 저는 고길동입니다.
안녕하세요, 저는 박희동입니다.
학번: 20250001
```

실습 2 상속 및 오버라이딩 [사람 ⇒ 교수, 학생]

- 지도교수 - 지도학생 introduce되는 기능 추가

```
class Person:

    def __init__(self, name):
        self.name = name

    def introduce(self):
        print(f'안녕하세요, 저는 {self.name}입니다.')

class Student(Person):

    def __init__(self, name, sid):
        super().__init__(name)   # 필요한 변수 입력해줘야 함
        self.sid = sid
        self.advisor = None

    def introduce(self):
        super().introduce()
        print(f'학번: {self.sid}')
        if self.advisor:
            print(f'지도교수: {self.advisor}')

class Professor(Person):

    def __init__(self,name):
        super().__init__(name)
        self.students = []

    def add_student(self, stu):
        self.students.append(stu)
```

```
        stu.advisor = self.name

    def introduce(self):
        super().introduce()
        if self.students:
            names = ','.join(stu.name for stu in self.students)
            print(f'지도학생: {names}')
        else:
            print(f'지도학생: 없음')

# 테스트 코드
p1 = Professor('박교수')
p2 = Professor('홍교수')
s1 = Student('김학생', 20191234)
s2 = Student('이학생', 20221234)
p1.add_student(s1)
p1.add_student(s2)
s1.introduce()
s2.introduce()
p1.introduce()
p2.introduce()
```

```
안녕하세요, 저는 김학생입니다.
학번: 20191234
지도교수: 박교수
안녕하세요, 저는 이학생입니다.
학번: 20221234
지도교수: 박교수
안녕하세요, 저는 박교수입니다.
지도학생: 김학생,이학생
안녕하세요, 저는 홍교수입니다.
지도학생: 없음
```

← 프로그래밍과 문제해결 · CSED101 목차로